

APOSTILA COMPLETA

MongoDB com Python

Do zero ao desenvolvimento de aplicações com PyMongo

Conteúdo: conceitos, instalação, MongoDB Atlas, BSON/JSON, banco de dados, coleções, CRUD, filtros, operadores de consulta, operadores de atualização, arrays, documentos aninhados, índices, agregação, validação, modelagem, transações, tratamento de erros, segurança, testes, APIs e projetos práticos.

Material didático em português • exemplos executáveis em Python

Referência técnica: documentação oficial do MongoDB e PyMongo. Consulte sempre a documentação da versão instalada para diferenças de API.

SUMÁRIO

1. Introdução ao MongoDB
2. Instalação e preparação do ambiente
3. MongoDB Atlas e conexão
4. Python + PyMongo
5. BSON, documentos e tipos de dados
6. Bancos, coleções e documentos
7. Inserção de documentos
8. Consultas e projeções
9. Operadores de comparação
10. Operadores lógicos
11. Operadores de elemento e avaliação
12. Arrays e documentos aninhados
13. Atualização de documentos
14. Operadores de atualização
15. Exclusão e contagem
16. Ordenação, paginação e distinct
17. Índices e desempenho
18. Agregação: pipelines e operadores
19. Modelagem de dados
20. Validação de schema
21. Transações e consistência
22. Tratamento de erros e boas práticas
23. Segurança e configuração
24. MongoDB com APIs Flask/FastAPI
25. Testes automatizados
26. Projeto completo: cadastro de produtos
27. Projeto completo: vendas e relatórios
28. Projeto completo: API de tarefas
29. Exercícios propostos
30. Gabarito comentado
31. Tabelas de referência rápida
32. Checklist final e próximos passos

1. INTRODUÇÃO AO MONGODB

MongoDB é um banco de dados orientado a documentos. Em vez de organizar os dados principalmente em tabelas e linhas, ele trabalha com coleções e documentos BSON. Isso permite representar objetos e estruturas aninhadas de maneira natural para aplicações Python.

O CRUD representa as quatro operações fundamentais: Create (criar), Read (ler), Update (atualizar) e Delete (excluir). No PyMongo, elas aparecem principalmente em métodos como `insert_one()`, `insert_many()`, `find_one()`, `find()`, `update_one()`, `update_many()`, `delete_one()` e `delete_many()`.

Objetivo da apostila

- Aprender MongoDB do nível iniciante ao intermediário/avançado.
- Usar Python com o driver oficial PyMongo.
- Entender filtros e operadores em vez de apenas decorar comandos.
- Construir consultas para documentos simples, aninhados e arrays.
- Criar pipelines de agregação para relatórios.
- Aplicar índices, validação, transações, segurança e testes.
- Construir projetos completos que sirvam de base para aplicações reais.

MongoDB não elimina a necessidade de modelagem. O fato de ser flexível não significa que devemos guardar qualquer estrutura de qualquer jeito. Uma aplicação bem projetada define nomes de campos, tipos, índices, regras de validação e padrões de consulta.

Como estudar

Execute os exemplos. Depois altere os valores, crie novos documentos e tente prever o resultado antes de rodar. A sequência recomendada é: conexão → inserção → consulta → atualização → exclusão → índices → agregação → projeto.

2. INSTALAÇÃO E PREPARAÇÃO DO AMBIENTE

Para estudar com Python, você precisa de Python 3, PyMongo e uma instância MongoDB local ou um cluster MongoDB Atlas. A instalação do driver é feita pelo pip.

Criando um ambiente virtual

```
python -m venv .venv
# Windows:
.venv\Scripts\activate

# Linux/macOS:
source .venv/bin/activate

python -m pip install --upgrade pip
python -m pip install pymongo
```

Use um ambiente virtual por projeto para evitar conflitos entre versões de pacotes.

Testando o PyMongo

```
import pymongo
print(pymongo.version)
```

Se o comando imprimir uma versão, o pacote está disponível no ambiente.

MongoDB local

Em uma instalação local, o servidor MongoDB precisa estar em execução antes que o Python tente se conectar. A URI local tradicionalmente usa o endereço 127.0.0.1 e uma porta do servidor.

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
print(client.list_database_names())
```

Diagnóstico rápido

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/", serverSelectionTimeoutMS=3000)

try:
    client.admin.command("ping")
    print("MongoDB respondeu ao ping.")
except Exception as erro:
    print("Não foi possível conectar:", erro)
finally:
    client.close()
```

3. MONGODB ATLAS E CONEXÃO

MongoDB Atlas é a opção hospedada em nuvem. O conceito central é o mesmo: você obtém uma URI de conexão, cria um MongoClient e seleciona banco e coleção.

Conexão com URI

```
from pymongo import MongoClient

URI = "mongodb+srv://USUARIO:SENHA@SEU_CLUSTER.mongodb.net/"
client = MongoClient(URI)

db = client["loja"]
produtos = db["produtos"]

print(db.name)
print(client.admin.command("ping"))
```

Nunca coloque usuário e senha diretamente no código que será publicado. Prefira variáveis de ambiente ou um gerenciador de segredos.

Variáveis de ambiente

```
import os
from pymongo import MongoClient

URI = os.environ["MONGODB_URI"]
client = MongoClient(URI)
db = client["loja"]

# Exemplo de variável no Windows PowerShell:
$env:MONGODB_URI="mongodb+srv://..."

# Linux/macOS:
export MONGODB_URI="mongodb+srv://..."
```

Encerrando a conexão

```
try:
    # operações no banco
    print(db.list_collection_names())
finally:
    client.close()
```

Em aplicações de longa duração, normalmente reutilize o MongoClient em vez de criar uma conexão nova para cada consulta.

4. PYTHON + PYMONGO

PyMongo é o driver oficial para trabalhar com MongoDB em Python. O fluxo mental é simples: MongoClient → database → collection → operação.

Hierarquia

Objeto	Exemplo	Função
MongoClient	client	Conexão com o servidor/cluster
Database	client['loja']	Representa um banco
Collection	db['produtos']	Conjunto de documentos
Document	{'nome': 'Teclado'}	Unidade de dados

```
client = MongoClient(URI)
db = client["loja"]
produtos = db["produtos"]
```

Padrão recomendado

```
from pymongo import MongoClient

def criar_cliente(uri: str) -> MongoClient:
    return MongoClient(uri)

client = criar_cliente(URI)
db = client["loja"]
produtos = db["produtos"]
```

Em projetos maiores, coloque a criação do cliente em um módulo de configuração e injete/reutilize o cliente.

Métodos essenciais

Categoria	PyMongo	Objetivo
Criar	insert_one / insert_many	Inserir documentos
Ler	find_one / find	Consultar
Atualizar	update_one / update_many	Alterar campos
Excluir	delete_one / delete_many	Remover
Agregação	aggregate	Processar e resumir dados
Índice	create_index	Otimizar consultas

5. BSON, DOCUMENTOS E TIPOS DE DADOS

MongoDB armazena documentos em BSON, uma representação binária semelhante conceitualmente ao JSON, mas com tipos adicionais. No Python, dicionários e listas são usados naturalmente para representar documentos e arrays.

Documento básico

```
produto = {
    "nome": "Teclado Mecânico",
    "preco": 249.90,
    "estoque": 12,
    "ativo": True
}

cliente = {
    "nome": "Maria",
    "email": "maria@example.com",
    "endereço": {
        "cidade": "São Paulo",
        "uf": "SP"
    },
    "telefones": ["11999990000", "1133334444"]
}
```

Tipos úteis

Python	MongoDB/BSON	Exemplo
dict	document/object	{'nome':'Ana'}
list	array	[10,20,30]
str	string	"São Paulo"
int/float	numeric	25 / 19.90
bool	boolean	True
None	null	None
datetime	date	datetime.now()
ObjectId	objectId	ObjectId('...')

ObjectId

```
from bson import ObjectId

_id = ObjectId()
print(_id)

doc = {"_id": _id, "nome": "Produto"}
```

Quando você não informa `_id` em uma inserção, o MongoDB normalmente gera um identificador para o documento.

Datas

```
from datetime import datetime, timezone

doc = {
    "evento": "login",
    "criado_em": datetime.now(timezone.utc)
}
```

Prefira datas com timezone explícito em aplicações distribuídas. Isso evita ambiguidades ao comparar eventos de diferentes regiões.

6. BANCOS, COLEÇÕES E DOCUMENTOS

MongoDB cria bancos e coleções conforme necessário quando uma operação de escrita é realizada. No Python, você pode obter referências sem necessariamente criar fisicamente o recurso naquele instante.

Selecionar banco e coleção

```
db = client["escola"]
alunos = db["alunos"]
```

Listar bancos e coleções

```
print(client.list_database_names())
print(db.list_collection_names())
```

Criar coleção explicitamente

```
db.create_collection("alunos")
```

A criação explícita é útil quando você precisa configurar opções da coleção, como validação.

Inserção simples

```
alunos.insert_one({
    "nome": "João",
    "idade": 20,
    "curso": "Python"
})
```

Convenção de nomes

Escolha nomes consistentes para coleções e campos. Evite alternar entre 'nome_cliente', 'clienteNome' e 'NomeCliente' sem uma razão. Consistência facilita consultas, índices e manutenção.

7. INSERÇÃO DE DOCUMENTOS

As operações de criação adicionam documentos a uma coleção. `insert_one()` insere um documento; `insert_many()` insere vários.

`insert_one()`

```
resultado = produtos.insert_one({
    "nome": "Mouse",
    "preco": 89.90,
    "estoque": 30
})

print(resultado.inserted_id)
```

`inserted_id` permite recuperar o identificador atribuído ao documento.

`insert_many()`

```
docs = [
    {"nome": "Monitor", "preco": 899.90, "estoque": 8},
    {"nome": "Webcam", "preco": 199.90, "estoque": 15},
    {"nome": "Headset", "preco": 159.90, "estoque": 22}
]

resultado = produtos.insert_many(docs)
print(resultado.inserted_ids)
```

Evitar duplicidade com índice único

```
produtos.create_index("sku", unique=True)

produtos.insert_one({
    "sku": "TEC-001",
    "nome": "Teclado",
    "preco": 249.90
})
```

Um índice `unique` impede valores duplicados no campo indexado, mas você deve tratar a exceção `DuplicateKeyError` no código.

Exercício rápido

Insira cinco produtos contendo `sku`, `nome`, `categoria`, `preço`, `estoque` e `ativo`. Depois imprima os IDs retornados.

8. CONSULTAS E PROJEÇÕES

`find_one()` retorna um documento ou `None`. `find()` retorna um cursor que pode ser percorrido. O primeiro argumento de `find/find_one` é o filtro; o segundo pode ser uma projeção.

`find_one()`

```
produto = produtos.find_one({"sku": "TEC-001"})
print(produto)

produto = produtos.find_one({"preco": {"$gt": 100}})
if produto:
    print(produto["nome"])
```

`find()`

```
for produto in produtos.find({"ativo": True}):
    print(produto["nome"], produto["preco"])
```

Projeção

```
for produto in produtos.find(
    {"categoria": "perifericos"},
    {"_id": 0, "nome": 1, "preco": 1}
):
    print(produto)
```

Na projeção, 1 inclui e 0 exclui. O campo `_id` é uma exceção comum: pode ser excluído explicitamente mesmo quando outros campos estão sendo incluídos.

Filtro vazio

```
todos = produtos.find({})
for doc in todos:
    print(doc)
```

Um filtro vazio seleciona todos os documentos. Tenha cuidado em operações de atualização e exclusão.

Consulta por campo aninhado

```
clientes.find_one({"endereco.cidade": "São Paulo"})
```

9. OPERADORES DE COMPARAÇÃO

Operadores de comparação aparecem dentro dos filtros. Eles são fundamentais para consultas por faixa de preço, idade, estoque, datas e outros valores.

Tabela de operadores

Operador	Significado	Exemplo
\$eq	igual	{'preco': {'\$eq': 100}}
\$ne	diferente	{'status': {'\$ne': 'cancelado'}}
\$gt	maior que	{'preco': {'\$gt': 100}}
\$gte	maior ou igual	{'idade': {'\$gte': 18}}
\$lt	menor que	{'preco': {'\$lt': 100}}
\$lte	menor ou igual	{'estoque': {'\$lte': 5}}
\$in	está em uma lista	{'uf': {'\$in': ['SP', 'RJ']}}
\$nin	não está na lista	{'uf': {'\$nin': ['SP', 'RJ']}}

Exemplos

```
# preço maior que 100
produtos.find({"preco": {"$gt": 100}})

# preço entre 100 e 500
produtos.find({
    "preco": {"$gte": 100, "$lte": 500}
})

# categorias selecionadas
produtos.find({
    "categoria": {"$in": ["perifericos", "audio"]}
})
```

Comparar datas

```
from datetime import datetime, timezone, timedelta

agora = datetime.now(timezone.utc)
inicio = agora - timedelta(days=7)

logs.find({"criado_em": {"$gte": inicio}})
```

Atalho para igualdade

Em muitos filtros, {'status': 'ativo'} equivale conceitualmente a {'status': {'\$eq': 'ativo'}}.

10. OPERADORES LÓGICOS

Os operadores lógicos combinam condições. Eles são especialmente úteis quando uma consulta precisa aceitar alternativas ou exigir várias regras.

Principais operadores

Operador	Uso
\$and	todas as condições devem ser verdadeiras
\$or	pelo menos uma condição deve ser verdadeira
\$nor	nenhuma das condições deve ser verdadeira
\$not	nega uma expressão de operador

\$and

```
produtos.find({
  "$and": [
    {"preco": {"$gte": 100}},
    {"estoque": {"$gt": 0}}
  ]
})
```

Na prática, condições sobre campos diferentes frequentemente podem ser escritas diretamente no mesmo documento de filtro.

\$or

```
produtos.find({
  "$or": [
    {"categoria": "audio"},
    {"preco": {"$lt": 50}}
  ]
})
```

\$nor

```
produtos.find({
  "$nor": [
    {"status": "cancelado"},
    {"estoque": 0}
  ]
})
```

\$not

```
produtos.find({
  "preco": {"$not": {"$gt": 1000}}
})
```

Exercício

Crie uma consulta que encontre produtos ativos com estoque maior que zero e preço abaixo de 500. Depois crie outra que encontre produtos da categoria audio OU com preço abaixo de 100.

11. OPERADORES DE ELEMENTO E AVALIAÇÃO

Além de comparar valores, MongoDB permite consultar existência de campos, tipos e padrões de texto.

\$exists

```
# documentos que possuem o campo telefone
clientes.find({"telefone": {"$exists": True}})

# documentos sem telefone
clientes.find({"telefone": {"$exists": False}})
```

\$type

```
# preço armazenado como número
produtos.find({"preco": {"$type": "number"}})
```

\$regex

```
# nomes que começam com "Tecl"
produtos.find({"nome": {"$regex": "^Tecl", "$options": "i"}})
```

Regex é útil, mas consultas textuais frequentes podem exigir uma estratégia de indexação/search apropriada.

\$expr

```
# compara campos dentro do mesmo documento
produtos.find({
  "$expr": {"$lt": ["$preco", "$preco_promocional"]}
})
```

Em \$expr, referências a campos são representadas por strings que começam com \$. O operador permite construir expressões envolvendo valores de campos.

\$jsonSchema como validação

Validação de schema é tratada em capítulo específico. Ela é diferente de usar \$jsonSchema em uma consulta; aqui o foco é reconhecer que MongoDB pode validar documentos de entrada.

12. ARRAYS E DOCUMENTOS ANINHADOS

Arrays são comuns em MongoDB: tags, telefones, itens de um pedido, avaliações e permissões. Documentos aninhados permitem agrupar dados relacionados.

Consulta em array

```
# tags contendo "python"
posts.find({"tags": "python"})
```

Uma consulta por um valor simples pode localizar documentos cujo array contém aquele valor.

\$all

```
posts.find({
  "tags": {"$all": ["python", "mongodb"]}
})
```

\$size

```
posts.find({
  "tags": {"$size": 3}
})
```

\$elemMatch

```
produtos.find({
  "avaliacoes": {
    "$elemMatch": {
      "nota": {"$gte": 4},
      "usuario": "Ana"
    }
  }
})
```

\$elemMatch é útil quando várias condições precisam ser satisfeitas pelo mesmo elemento de um array de documentos.

Atualizando arrays

```
produtos.update_one(
  {"sku": "TEC-001"},
  {"$push": {"tags": "promocao"}}
)

produtos.update_one(
  {"sku": "TEC-001"},
  {"$addToSet": {"tags": "promocao"}}
)
```

\$push adiciona; \$addToSet evita adicionar o mesmo valor novamente ao array.

Campos aninhados

```
cliente = {
  "nome": "Carlos",
  "endereco": {
    "cidade": "Guarulhos",
    "uf": "SP"
  }
}

clientes.find({"endereco.uf": "SP"})
```

13. ATUALIZAÇÃO DE DOCUMENTOS

As atualizações usam um filtro para localizar documentos e um documento de atualização para definir as alterações. `update_one` altera no máximo um documento; `update_many` pode alterar vários.

update_one()

```
resultado = produtos.update_one(
    {"sku": "TEC-001"},
    {"$set": {"preco": 219.90}}
)

print(resultado.matched_count)
print(resultado.modified_count)
```

update_many()

```
resultado = produtos.update_many(
    {"categoria": "audio"},
    {"$set": {"ativo": True}}
)

print("Encontrados:", resultado.matched_count)
print("Alterados:", resultado.modified_count)
```

replace_one()

```
resultado = produtos.replace_one(
    {"sku": "TEC-001"},
    {
        "sku": "TEC-001",
        "nome": "Teclado Novo",
        "preco": 299.90,
        "estoque": 20
    }
)
```

`replace_one` substitui o documento inteiro, preservando o `_id` quando aplicável. Para alterar campos específicos, prefira operadores como `$set`.

upsert

```
produtos.update_one(
    {"sku": "NOVO-001"},
    {"$set": {"nome": "Produto Novo", "preco": 49.90}},
    upsert=True
)
```

Com `upsert=True`, se nenhum documento corresponder ao filtro, o MongoDB pode criar um novo documento a partir do filtro e da atualização.

Retornar o documento alterado

```
from pymongo import ReturnDocument

doc = produtos.find_one_and_update(
    {"sku": "TEC-001"},
    {"$inc": {"estoque": -1}},
    return_document=ReturnDocument.AFTER
)

print(doc)
```

14. OPERADORES DE ATUALIZAÇÃO

Os operadores de atualização permitem alterar campos, números, arrays e estruturas sem substituir o documento inteiro.

Campos

Operador	Função
\$set	define/substitui valor de um campo
\$unset	remove campo
\$rename	renomeia campo
\$setOnInsert	define campo somente quando ocorre upsert com inserção

```
produtos.update_one(
    {"sku": "TEC-001"},
    {
        "$set": {"ativo": True},
        "$unset": {"campo_antigo": ""},
        "$rename": {"nome_produto": "nome"}
    }
)
```

Operadores numéricos

Operador	Função
\$inc	incrementa ou decrementa
\$mul	multiplica
\$min	define o menor valor entre atual e novo
\$max	define o maior valor entre atual e novo

```
# aumentar preço em 10%
produtos.update_many(
    {"categoria": "audio"},
    {"$mul": {"preco": 1.10}}
)

# diminuir estoque em 1
produtos.update_one(
    {"sku": "TEC-001"},
    {"$inc": {"estoque": -1}}
)
```

Arrays

Operador	Função
\$push	adiciona item ao array
\$addToSet	adiciona somente se ainda não existir
\$pop	remove primeiro ou último item
\$pull	remove itens que atendem a uma condição
\$pullAll	remove valores especificados
\$each	insere vários valores junto a \$push
\$position	controla posição com \$push
\$slice	limita elementos com \$push

```
produtos.update_one(
```

```
        {"sku": "TEC-001"},
        {"$push": {"tags": {"$each": ["oferta", "gaming"]}}}
    )

produtos.update_one(
    {"sku": "TEC-001"},
    {"$pull": {"tags": "oferta"}}
)
```

Operadores posicionais

```
# atualizar um elemento de um array conforme o filtro
pedidos.update_one(
    {"_id": pedido_id, "itens.sku": "TEC-001"},
    {"$set": {"itens.$.quantidade": 3}}
)
```

Arrays complexos podem exigir operadores posicionais, `$[identificador]` e `array_filters`. Use filtros precisos para evitar alterações inesperadas.

15. EXCLUSÃO E CONTAGEM

Exclusão é uma operação destrutiva. Sempre teste o filtro com `find()` antes de chamar `delete_many()` quando estiver trabalhando com dados importantes.

`delete_one()`

```
resultado = produtos.delete_one({"sku": "TEC-001"})
print(resultado.deleted_count)
```

`delete_many()`

```
resultado = produtos.delete_many({
    "ativo": False,
    "estoque": 0
})
print(resultado.deleted_count)
```

Nunca use `delete_many({})` sem ter certeza de que deseja apagar todos os documentos da coleção.

Contagem

```
total = produtos.count_documents({})
ativos = produtos.count_documents({"ativo": True})
print(total, ativos)
```

`distinct`

```
categorias = produtos.distinct("categoria")
print(categorias)
```

`distinct` retorna os valores distintos de um campo. Para análises mais elaboradas, use agregação.

Exercício de segurança

Escreva primeiro uma consulta `find()` que identifique exatamente os documentos que seriam excluídos. Só depois transforme o filtro em `delete_many()`.

16. ORDENAÇÃO, PAGINAÇÃO E DISTINCT

Ordenar e paginar dados é essencial em aplicações web. Em PyMongo, `sort()`, `skip()` e `limit()` são usados sobre cursores.

`sort()`

```
# menor para maior
cursor = produtos.find().sort("preco", 1)

# maior para menor
cursor = produtos.find().sort("preco", -1)
```

`limit()`

```
mais_baratos = (
    produtos.find({"ativo": True})
    .sort("preco", 1)
    .limit(10)
)

for doc in mais_baratos:
    print(doc["nome"], doc["preco"])
```

`skip()` e paginação

```
pagina = 3
por_pagina = 10

cursor = (
    produtos.find({})
    .sort("_id", 1)
    .skip((pagina - 1) * por_pagina)
    .limit(por_pagina)
)
```

Para conjuntos muito grandes, paginação por cursor/range pode ser mais eficiente do que grandes valores de skip.

Filtro + projeção + ordenação

```
cursor = (
    produtos.find(
        {"preco": {"$gte": 100}},
        {"_id": 0, "nome": 1, "preco": 1}
    )
    .sort("preco", -1)
    .limit(20)
)
```

17. ÍNDICES E DESEMPENHO

Índices permitem que o MongoDB encontre dados com mais eficiência para determinadas consultas. Eles também têm custo: ocupam espaço e tornam algumas escritas mais caras. O índice correto depende dos padrões de consulta.

Criando índices

```
# índice simples
produtos.create_index("sku")

# índice único
produtos.create_index("sku", unique=True)

# índice composto
produtos.create_index([
    ("categoria", 1),
    ("preco", -1)
])
```

Listar e remover índices

```
print(list(produtos.list_indexes()))

produtos.drop_index("sku_1")
```

Índices e consultas

Se a aplicação consulta frequentemente por sku, um índice em sku pode ser apropriado. Se consulta por categoria e ordena por preço, um índice composto pode atender melhor, dependendo da consulta e da cardinalidade dos dados.

```
produtos.create_index([
    ("categoria", 1),
    ("preco", -1)
])
```

explain()

```
explicacao = produtos.find(
    {"categoria": "audio"}
).explain()

print(explicacao)
```

Use explain para investigar o plano de execução. Procure entender se a consulta usa um índice adequado ou faz uma varredura completa quando isso é indesejado.

Regra prática

Não crie índices indiscriminadamente. Comece pelos padrões de consulta reais, meça com explain e considere custo de escrita e memória.

18. AGREGAÇÃO: PIPELINES E OPERADORES

Aggregation pipeline é o mecanismo para filtrar, transformar, agrupar, ordenar e resumir documentos. Em Python, o pipeline é normalmente uma lista de dicionários passada a `collection.aggregate()`.

Pipeline básico

```
pipeline = [
    {"$match": {"ativo": True}},
    {"$group": {
        "_id": "$categoria",
        "total": {"$sum": 1},
        "media_preco": {"$avg": "$preco"}
    }},
    {"$sort": {"total": -1}}
]

for item in produtos.aggregate(pipeline):
    print(item)
```

Estágios principais

Estágio	Finalidade
\$match	filtra documentos
\$project	seleciona/cria campos
\$set / \$addFields	adiciona ou calcula campos
\$unset	remove campos
\$group	agrupa e calcula
\$sort	ordena
\$limit	limita
\$skip	pula documentos
\$unwind	desmonta um array em documentos
\$lookup	combina dados entre coleções
\$count	conta documentos
\$facet	executa múltiplos pipelines em paralelo

\$project

```
pipeline = [
    {"$project": {
        "_id": 0,
        "nome": 1,
        "preco": 1,
        "preco_com_desconto": {
            "$multiply": ["$preco", 0.90]
        }
    }}
]
```

\$group

```
pipeline = [
    {"$group": {
        "_id": "$categoria",
        "quantidade": {"$sum": 1},
        "estoque_total": {"$sum": "$estoque"},
        "preco_medio": {"$avg": "$preco"},
        "preco_maximo": {"$max": "$preco"},
        "preco_minimo": {"$min": "$preco"}
    }}
]
```

```
    }}
  ]
}
```

\$unwind

```
pipeline = [
    {"$unwind": "$tags"},
    {"$group": {
        "_id": "$tags",
        "total": {"$sum": 1}
    }},
    {"$sort": {"total": -1}}
]
```

\$lookup

```
pipeline = [
    {"$lookup": {
        "from": "categorias",
        "localField": "categoria_id",
        "foreignField": "_id",
        "as": "categoria"
    }},
    {"$unwind": "$categoria"}
]
```

\$lookup pode ser útil, mas MongoDB também permite modelagem com documentos embutidos. Escolha conforme acesso, tamanho, atualização e relacionamento dos dados.

Expressões comuns

Operador	Exemplo de uso
\$sum	somar valores ou contar com 1
\$avg	média
\$min/\$max	mínimo/máximo
\$multiply	multiplicação
\$add/\$subtract	operações aritméticas
\$cond	condicional
\$ifNull	valor alternativo para nulo/ausente
\$concat	concatenar strings
\$toString	converter para string

Relatório de vendas

```
pipeline = [
    {"$match": {"status": "pago"}},
    {"$unwind": "$itens"},
    {"$group": {
        "_id": "$itens.sku",
        "quantidade": {"$sum": "$itens.quantidade"},
        "faturamento": {
            "$sum": {
                "$multiply": [
                    "$itens.quantidade",
                    "$itens.preco"
                ]
            }
        }
    }},
    {"$sort": {"faturamento": -1}}
]
```

```
for linha in pedidos.aggregate(pipeline):
    print(linha)
```

19. MODELAGEM DE DADOS

MongoDB permite embutir documentos ou referenciar outros documentos. A decisão deve partir de como a aplicação lê e modifica os dados.

Embutir

```
pedido = {  
    "cliente": {"nome": "Ana", "email": "ana@example.com"},  
    "itens": [  
        {"sku": "A1", "quantidade": 2, "preco": 50},  
        {"sku": "B2", "quantidade": 1, "preco": 120}  
    ]  
}
```

Embutir é conveniente quando os dados são acessados juntos e o crescimento do documento permanece controlado.

Referenciar

```
pedido = {  
    "cliente_id": cliente_id,  
    "itens": [  
        {"produto_id": produto_id, "quantidade": 2}  
    ]  
}
```

Referências podem ser apropriadas quando entidades são compartilhadas, crescem independentemente ou são consultadas separadamente.

Perguntas antes de modelar

Quais consultas são mais frequentes?

Quais dados são sempre exibidos juntos?

Qual é o tamanho máximo esperado de arrays?

Quais dados mudam juntos?

Há necessidade de consistência transacional entre documentos?

Qual é o custo de duplicar um pequeno conjunto de dados?

Anti-padrões

Arrays que crescem indefinidamente dentro de um único documento.

Duplicação de dados sem estratégia de atualização.

Modelagem baseada apenas no diagrama relacional tradicional, ignorando os padrões de acesso da aplicação.

20. VALIDAÇÃO DE SCHEMA

Embora MongoDB seja flexível, você pode aplicar regras de validação para impedir documentos incompatíveis com a estrutura esperada.

Criando coleção com validação

```
validator = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["nome", "preco"],
        "properties": {
            "nome": {"bsonType": "string"},
            "preco": {"bsonType": ["double", "int", "long", "decimal"]}
        }
    }
}

db.create_collection(
    "produtos_validos",
    validator=validator
)
```

Por que validar?

Evita que diferentes partes da aplicação criem campos incompatíveis.

Documenta expectativas de dados no próprio banco.

Reduz erros difíceis de detectar posteriormente.

Pode ser combinado com validações no Python.

Validação no Python

```
def validar_produto(produto):
    if not produto.get("nome"):
        raise ValueError("Nome obrigatório")
    if produto.get("preco", 0) < 0:
        raise ValueError("Preço não pode ser negativo")
```

A validação da aplicação não substitui necessariamente a validação no banco; as duas camadas podem ter objetivos complementares.

21. TRANSAÇÕES E CONSISTÊNCIA

Transações permitem agrupar operações que devem ser confirmadas ou desfeitas em conjunto. No PyMongo, elas usam sessões. A documentação oficial recomenda reutilizar o MongoClient e criar sessões para as transações necessárias.

Exemplo com with_transaction

```
from pymongo import MongoClient

client = MongoClient(URI)
db = client["banco"]
contas = db["contas"]

def transferir(session, origem, destino, valor):
    contas.update_one(
        {"_id": origem},
        {"$inc": {"saldo": -valor}},
        session=session
    )
    contas.update_one(
        {"_id": destino},
        {"$inc": {"saldo": valor}},
        session=session
    )

with client.start_session() as session:
    session.with_transaction(
        lambda s: transferir(s, "A", "B", 100)
    )
```

Uma transação só deve ser usada quando a atomicidade entre múltiplas operações realmente for necessária. Muitas operações de um único documento já são atômicas no nível do documento.

Controle manual

```
with client.start_session() as session:
    session.start_transaction()
    try:
        contas.update_one(
            {"_id": "A"},
            {"$inc": {"saldo": -100}},
            session=session
        )
        contas.update_one(
            {"_id": "B"},
            {"$inc": {"saldo": 100}},
            session=session
        )
        session.commit_transaction()
    except Exception:
        session.abort_transaction()
        raise
```

Quando pensar em transação

Transferência entre contas.

Atualização coordenada de documentos que precisa ser tudo-ou-nada.

Operações de negócio em múltiplas coleções que não podem ficar parcialmente aplicadas.

22. TRATAMENTO DE ERROS E BOAS PRÁTICAS

Aplicações reais precisam tratar erros de conexão, duplicidade, validação e operações que não encontraram documentos.

Exceções comuns

```
from pymongo.errors import (
    PyMongoError,
    DuplicateKeyError,
    ServerSelectionTimeoutError
)

try:
    produtos.insert_one({"sku": "A1"})
except DuplicateKeyError:
    print("SKU já existe.")
except ServerSelectionTimeoutError:
    print("Servidor MongoDB não respondeu.")
except PyMongoError as erro:
    print("Erro do MongoDB:", erro)
```

Não engolir exceções

Evite usar `except Exception: pass`. Isso transforma falhas reais em comportamentos silenciosos. Registre o erro, trate o caso esperado ou relance a exceção.

Separação de responsabilidades

Um módulo deve cuidar da conexão/configuração.

Outro pode conter repositórios ou funções de acesso a dados.

A camada de serviço deve concentrar regras de negócio.

A API deve validar entrada e formatar respostas.

Exemplo de repositório

```
class ProdutoRepository:
    def __init__(self, collection):
        self.collection = collection

    def buscar_por_sku(self, sku):
        return self.collection.find_one({"sku": sku})

    def criar(self, produto):
        return self.collection.insert_one(produto)

    def atualizar_preco(self, sku, preco):
        return self.collection.update_one(
            {"sku": sku},
            {"$set": {"preco": preco}}
        )
```

Boas práticas de produção

Reutilizar MongoClient.

Usar timeouts apropriados.

Guardar URI e credenciais fora do código.

Usar índices baseados em consultas reais.

Validar entradas.

Registrar erros sem expor segredos.

Testar operações destrutivas com dados controlados.

23. SEGURANÇA E CONFIGURAÇÃO

Segurança começa por credenciais, permissões e configuração de rede. O código Python deve receber apenas os privilégios necessários.

Nunca faça isso

```
# NÃO publique isto em Git:  
URI = "mongodb+srv://admin:MINHA_SENHA@cluster..."
```

Também não registre a URI completa em logs, porque ela pode conter credenciais.

Variáveis de ambiente

```
import os  
  
URI = os.getenv("MONGODB_URI")  
if not URI:  
    raise RuntimeError("MONGODB_URI não configurada")
```

Princípio do menor privilégio

Use um usuário do banco com apenas as permissões necessárias para a aplicação. Uma aplicação de leitura não precisa necessariamente de permissão de escrita administrativa.

TLS e Atlas

Use a URI fornecida pelo ambiente de hospedagem e as configurações recomendadas pelo provedor. Não desabilite verificações de certificado apenas para contornar erros de conexão.

Segredos

Em desenvolvimento, arquivos .env podem ser usados com cuidado e devem ficar fora do repositório quando contêm segredos. Em produção, prefira serviços de secrets management.

24. MONGODB COM APIs FLASK/FASTAPI

MongoDB combina bem com APIs Python. A ideia é manter o cliente do banco reutilizável e transformar documentos em respostas JSON com cuidado, especialmente no caso de ObjectId e datetime.

Exemplo mínimo com Flask

```
from flask import Flask, jsonify
from pymongo import MongoClient
import os

app = Flask(__name__)
client = MongoClient(os.environ["MONGODB_URI"])
db = client["loja"]
produtos = db["produtos"]

@app.get("/produtos")
def listar_produtos():
    docs = list(produtos.find({}, {"_id": 0}))
    return jsonify(docs)

if __name__ == "__main__":
    app.run(debug=True)
```

ObjectId e JSON

```
from bson import ObjectId

doc = produtos.find_one({"_id": ObjectId(id_texto)})

resposta = {
    "id": str(doc["_id"]),
    "nome": doc["nome"]
}
```

ObjectId não é um tipo JSON nativo. Em APIs, uma estratégia comum é serializá-lo como string.

FastAPI — estrutura conceitual

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/produtos")
def listar():
    return {"mensagem": "consultar MongoDB aqui"}
```

Em projetos maiores, combine Pydantic para validação de entrada/saída com uma camada de acesso ao MongoDB.

Paginação de API

```
@app.get("/produtos")
def listar(page: int = 1, limit: int = 20):
    skip = (page - 1) * limit
    docs = list(
        produtos.find(
            {},
            {"_id": 0}
        ).sort("nome", 1).skip(skip).limit(limit)
    )
    return {"page": page, "limit": limit, "items": docs}
```

25. TESTES AUTOMATIZADOS

Testar a camada de acesso ao MongoDB evita regressões. O ideal é que os testes sejam isolados dos dados de produção.

Pytest — exemplo

```
def test_criar_produto(produtos):
    resultado = produtos.insert_one({
        "sku": "TEST-001",
        "nome": "Produto de Teste",
        "preco": 10
    })

    assert resultado.inserted_id is not None
```

Fixture

```
import pytest
from pymongo import MongoClient

@pytest.fixture
def produtos():
    client = MongoClient("mongodb://localhost:27017/")
    db = client["loja_testes"]
    collection = db["produtos"]
    collection.delete_many({})

    yield collection

    collection.delete_many({})
    client.close()
```

Para suítes maiores, considere estratégias específicas para banco de testes, isolamento e execução em CI.

O que testar

Criação de documento.

Consulta por identificador e por campos.

Atualização com operador correto.

Exclusão com filtro correto.

Duplicidade de índice unique.

Validação de entrada.

Casos de documento inexistente.

Erros de conexão, quando aplicável.

26. PROJETO COMPLETO: CADASTRO DE PRODUTOS

Este projeto reúne conexão, índice, CRUD, filtros e funções reutilizáveis. É uma base para um pequeno sistema de estoque.

Estrutura

```
loja_mongodb/  
├─ app.py  
├─ banco.py  
├─ produtos.py  
└─ .env
```

banco.py

```
import os  
from pymongo import MongoClient  
  
client = MongoClient(os.environ["MONGODB_URI"])  
db = client["loja"]  
produtos = db["produtos"]  
  
produtos.create_index("sku", unique=True)
```

produtos.py

```
from banco import produtos  
  
def criar(dados):  
    return produtos.insert_one(dados)  
  
def buscar(sku):  
    return produtos.find_one({"sku": sku})  
  
def listar(categoria=None):  
    filtro = {}  
    if categoria:  
        filtro["categoria"] = categoria  
    return list(produtos.find(filtro).sort("nome", 1))  
  
def alterar_preco(sku, novo_preco):  
    return produtos.update_one(  
        {"sku": sku},  
        {"$set": {"preco": novo_preco}}  
    )  
  
def remover(sku):  
    return produtos.delete_one({"sku": sku})
```

app.py

```
from produtos import criar, buscar, listar  
  
criar(  
    "sku": "MOUSE-001",  
    "nome": "Mouse Gamer",  
    "categoria": "perifericos",  
    "preco": 149.90,  
    "estoque": 25,  
    "ativo": True  
)  
  
print(buscar("MOUSE-001"))  
  
for item in listar("perifericos"):  
    print(item)
```

Melhorias

Adicionar validação.

Adicionar atualização de estoque.

Adicionar endpoint HTTP.

Adicionar paginação.

Criar relatório de vendas com `aggregate()`.

27. PROJETO COMPLETO: VENDAS E RELATÓRIOS

Vamos modelar pedidos com itens embutidos e produzir relatórios usando \$unwind, \$group, \$sum e \$multiply.

Documento de pedido

```
{
  "cliente_id": "C001",
  "data": "2026-10-01",
  "status": "pago",
  "itens": [
    {"sku": "A1", "quantidade": 2, "preco": 50.0},
    {"sku": "B2", "quantidade": 1, "preco": 120.0}
  ]
}
```

Faturamento por SKU

```
pipeline = [
    {"$match": {"status": "pago"}},
    {"$unwind": "$itens"},
    {"$group": {
        "_id": "$itens.sku",
        "unidades": {"$sum": "$itens.quantidade"},
        "faturamento": {
            "$sum": {
                "$multiply": [
                    "$itens.quantidade",
                    "$itens.preco"
                ]
            }
        }
    }},
    {"$sort": {"faturamento": -1}}
]

resultado = list(pedidos.aggregate(pipeline))
for linha in resultado:
    print(linha)
```

Faturamento por cliente

```
pipeline = [
    {"$match": {"status": "pago"}},
    {"$unwind": "$itens"},
    {"$group": {
        "_id": "$cliente_id",
        "total": {
            "$sum": {
                "$multiply": [
                    "$itens.quantidade",
                    "$itens.preco"
                ]
            }
        }
    }},
    {"$sort": {"total": -1}}
]
```

Exercício

Crie um relatório que mostre: quantidade de pedidos pagos, faturamento total, ticket médio e faturamento por cliente.

28. PROJETO COMPLETO: API DE TAREFAS

Uma API de tarefas é pequena o suficiente para estudar CRUD, mas rica o bastante para praticar filtros, índices, datas e paginação.

Modelo

```
{
  "titulo": "Estudar MongoDB",
  "descricao": "Revisar operadores",
  "concluida": False,
  "prioridade": 3,
  "tags": ["mongodb", "python"],
  "criada_em": "datetime"
}
```

Consultas úteis

```
# tarefas abertas
tarefas.find({"concluida": False})

# prioridade alta
tarefas.find({"prioridade": {"$gte": 4}})

# com uma tag
tarefas.find({"tags": "mongodb"})
```

Atualizações

```
tarefas.update_one(
    {"_id": tarefa_id},
    {"$set": {"concluida": True}}
)

tarefas.update_one(
    {"_id": tarefa_id},
    {"$addToSet": {"tags": "importante"}}
)
```

Índices

```
tarefas.create_index([
    ("concluida", 1),
    ("prioridade", -1)
])
```

Desafio

Implemente endpoints para criar, listar, atualizar, concluir e excluir tarefas. Acrescente paginação e filtro por tag.

29. EXERCÍCIOS PROPOSTOS

Resolva os exercícios sem consultar o gabarito imediatamente. Crie uma coleção de treino chamada `exercicios_mongodb`.

Nível 1 — fundamentos

1. Conecte ao MongoDB e execute ping.
2. Crie a coleção `alunos` e insira cinco alunos.
3. Consulte todos os alunos.
4. Consulte apenas um aluno por nome.
5. Faça uma projeção retornando apenas nome e idade.
6. Conte quantos alunos existem.

Nível 2 — operadores

7. Encontre produtos com preço maior que 100.
8. Encontre produtos entre 100 e 500.
9. Encontre produtos cuja categoria esteja em uma lista.
10. Encontre documentos que possuam telefone.
11. Use `$or` para consultar duas condições alternativas.
12. Use `regex` para localizar nomes iniciados por uma letra.

Nível 3 — atualização

13. Aumente o preço de uma categoria em 10%.
14. Diminua o estoque de um produto em 1.
15. Adicione uma tag com `$addToSet`.
16. Remova uma tag com `$pull`.
17. Faça um `upsert` por sku.
18. Renomeie um campo.

Nível 4 — agregação

19. Calcule preço médio por categoria.
20. Calcule estoque total por categoria.
21. Desmonte um array de itens com `$unwind`.
22. Calcule faturamento por SKU.
23. Use `$lookup` para relacionar pedidos e clientes.
24. Crie um pipeline com `$match`, `$group` e `$sort`.

Nível 5 — projeto

25. Construa uma API de produtos.
26. Adicione índice `unique` para SKU.

- 27. Adicione paginação.
- 28. Adicione testes.
- 29. Adicione relatório de vendas.
- 30. Proteja a URI usando variável de ambiente.

30. GABARITO COMENTADO

Abaixo estão soluções-modelo. Elas não são as únicas respostas possíveis; o objetivo é comparar sua abordagem e entender a lógica.

Exercícios 1-6

```
from pymongo import MongoClient

client = MongoClient("mongodb://localhost:27017/")
db = client["exercicios_mongodb"]
alunos = db["alunos"]

client.admin.command("ping")

alunos.delete_many({})
alunos.insert_many([
    {"nome": "Ana", "idade": 20},
    {"nome": "Bruno", "idade": 22},
    {"nome": "Carlos", "idade": 19},
    {"nome": "Diana", "idade": 25},
    {"nome": "Eduardo", "idade": 21},
])

print(list(alunos.find({})))
print(alunos.find_one({"nome": "Ana"}))
print(list(alunos.find({}, {"_id": 0, "nome": 1, "idade": 1})))
print(alunos.count_documents({}))
```

Exercícios 7-12

```
produtos.find({"preco": {"$gt": 100}})
produtos.find({"preco": {"$gte": 100, "$lte": 500}})
produtos.find({"categoria": {"$in": ["audio", "perifericos"]}})
clientes.find({"telefone": {"$exists": True}})
produtos.find({
    "$or": [
        {"categoria": "audio"},
        {"preco": {"$lt": 50}}
    ]
})
produtos.find({"nome": {"$regex": "^M", "$options": "i"}})
```

Exercícios 13-18

```
produtos.update_many(
    {"categoria": "audio"},
    {"$mul": {"preco": 1.10}}
)

produtos.update_one(
    {"sku": "A1"},
    {"$inc": {"estoque": -1}}
)

produtos.update_one(
    {"sku": "A1"},
    {"$addToSet": {"tags": "oferta"}}
)

produtos.update_one(
    {"sku": "A1"},
    {"$pull": {"tags": "oferta"}}
)

produtos.update_one(
    {"sku": "NOVO"},
    {"$set": {"nome": "Novo", "preco": 10}},
    upsert=True
)

produtos.update_many(
    {},
    {"$rename": {"descricao_curta": "descricao"}}
```

)

Exercícios 19-24

```
# preço médio por categoria
[
  {"$group": {
    "_id": "$categoria",
    "media": {"$avg": "$preco"}
  }}
]

# estoque total
[
  {"$group": {
    "_id": "$categoria",
    "estoque": {"$sum": "$estoque"}
  }}
]

# faturamento
[
  {"$unwind": "$itens"},
  {"$group": {
    "_id": "$itens.sku",
    "faturamento": {
      "$sum": {
        "$multiply": [
          "$itens.quantidade",
          "$itens.preco"
        ]
      }
    }
  }}
],
{"$sort": {"faturamento": -1}}
]
```

31. TABELAS DE REFERÊNCIA RÁPIDA

Use este capítulo como folha de consulta durante seus estudos e projetos.

CRUD PyMongo

Ação	Método
Inserir 1	collection.insert_one(doc)
Inserir vários	collection.insert_many(docs)
Buscar 1	collection.find_one(filtro)
Buscar vários	collection.find(filtro)
Atualizar 1	collection.update_one(filtro, update)
Atualizar vários	collection.update_many(filtro, update)
Substituir	collection.replace_one(filtro, doc)
Excluir 1	collection.delete_one(filtro)
Excluir vários	collection.delete_many(filtro)
Contar	collection.count_documents(filtro)
Distintos	collection.distinct(campo)
Agregação	collection.aggregate(pipeline)
Índice	collection.create_index(campo)

Filtros

Grupo	Operadores
Comparação	\$eq \$ne \$gt \$gte \$lt \$lte \$in \$nin
Lógica	\$and \$or \$nor \$not
Elemento	\$exists \$type
Avaliação	\$expr \$regex \$text \$where*
Array	\$all \$elemMatch \$size

* \$where tem implicações de segurança/desempenho e não deve ser usado sem entender as restrições do ambiente. Prefira operadores nativos quando possível.

Atualização

Categoria	Operadores
Campos	\$set \$unset \$rename \$setOnInsert
Números	\$inc \$mul \$min \$max
Arrays	\$push \$addToSet \$pop \$pull \$pullAll
Posicional	\$ \$[] \$[identifier]

Agregação

Necessidade	Estágios/expressões comuns
Filtrar	\$match
Selecionar/calcular	\$project \$set \$unset
Agrupar	\$group
Ordenar	\$sort
Paginar	\$skip \$limit
Arrays	\$unwind
Relacionar	\$lookup
Contar	\$count
Múltiplas análises	\$facet

Métodos de cursor

```
cursor.sort("campo", 1)
cursor.limit(20)
cursor.skip(40)
```

```
for doc in cursor:
    print(doc)
```

32. CHECKLIST FINAL E PRÓXIMOS PASSOS

Ao terminar esta apostila, você deve ser capaz de construir uma aplicação Python que use MongoDB com segurança e consultas bem estruturadas.

Checklist

- ☐ Instalar e importar PyMongo.
- ☐ Conectar a MongoDB local ou Atlas.
- ☐ Selecionar database e collection.
- ☐ Inserir um e vários documentos.
- ☐ Consultar com `find_one` e `find`.
- ☐ Usar projeções.
- ☐ Usar operadores de comparação.
- ☐ Usar operadores lógicos.
- ☐ Consultar arrays e documentos aninhados.
- ☐ Atualizar com `$set`, `$inc`, `$push` e `$pull`.
- ☐ Usar `upsert`.
- ☐ Excluir com segurança.
- ☐ Ordenar e paginar.
- ☐ Criar e analisar índices.
- ☐ Criar pipelines de agregação.
- ☐ Modelar dados embutidos e referenciados.
- ☐ Validar documentos.
- ☐ Trabalhar com transações.
- ☐ Tratar exceções.
- ☐ Proteger credenciais.
- ☐ Integrar MongoDB a uma API.
- ☐ Criar testes automatizados.

Projetos para continuar

1. Sistema de estoque com entrada e saída de produtos.
2. E-commerce com carrinho e pedidos.
3. API de clientes e contatos.
4. Sistema de tarefas com usuários e permissões.
5. Dashboard de vendas usando aggregation.
6. Sistema de avaliações de produtos.
7. Catálogo de músicas com artistas, álbuns e faixas.

Referências oficiais

MongoDB Manual: <https://www.mongodb.com/docs/>

MongoDB + Python/PyMongo:

<https://www.mongodb.com/docs/languages/python/pymongo-driver/current/>

PyMongo API: <https://pymongo.readthedocs.io/>

MongoDB University — Python:

<https://learn.mongodb.com/learning-paths/mongodb-python-developer-path>

As APIs podem evoluir. Para projetos novos, confira a documentação correspondente à versão do MongoDB Server e do PyMongo instalados.

Conclusão

MongoDB com Python fica muito mais simples quando você entende o modelo mental: documento → filtro → operador → resultado. Em seguida, agregação e índices permitem transformar consultas simples em sistemas completos.

Não tente decorar todos os operadores. Aprenda a reconhecer o problema: comparar, combinar condições, atualizar, trabalhar com arrays, agrupar ou ordenar. Depois consulte a tabela de referência e a documentação.

APÊNDICE A — COMANDOS MONGOSH E EQUIVALENTES PYTHON

Embora a aplicação use PyMongo, conhecer o mongosh ajuda a testar consultas rapidamente e entender a sintaxe do MongoDB Query Language.

Inserção

```
// mongosh
db.produtos.insertOne({
  nome: "Mouse",
  preco: 99.90
})

# Python
produtos.insert_one({
  "nome": "Mouse",
  "preco": 99.90
})
```

Consulta

```
// mongosh
db.produtos.find({ preco: { $gt: 100 } })

# Python
produtos.find({"preco": {"$gt": 100}})
```

Atualização

```
// mongosh
db.produtos.updateOne(
  { sku: "A1" },
  { $inc: { estoque: -1 } }
)

# Python
produtos.update_one(
  {"sku": "A1"},
  {"$inc": {"estoque": -1}}
)
```

Exclusão

```
// mongosh
db.produtos.deleteOne({ sku: "A1" })

# Python
produtos.delete_one({"sku": "A1"})
```

Agregação

```
// mongosh
db.produtos.aggregate([
  { $group: {
    _id: "$categoria",
    media: { $avg: "$preco" }
  }}
])

# Python
produtos.aggregate([
  {"$group": {
    "_id": "$categoria",
    "media": {"$avg": "$preco"}
  }}
])
```

APÊNDICE B — DICAS DE DEPURAÇÃO

Quando uma consulta não retorna o que você esperava, investigue por etapas em vez de alterar várias coisas de uma vez.

Passo 1 — confirme a conexão

```
client.admin.command("ping")
```

Passo 2 — veja documentos reais

```
for doc in produtos.find({}).limit(5):  
    print(doc)
```

Passo 3 — teste somente o filtro

```
filtro = {"categoria": "audio"}  
print(produtos.count_documents(filtro))  
print(produtos.find_one(filtro))
```

Passo 4 — teste projeção e ordenação separadamente

```
cursor = produtos.find(  
    filtro,  
    {"_id": 0, "nome": 1, "preco": 1}  
)  
cursor = cursor.sort("preco", -1)  
print(list(cursor.limit(5)))
```

Passo 5 — use explain

```
exp = produtos.find(filtro).explain()  
print(exp)
```

Erros frequentes

Nome de campo incorreto.

Tipo diferente do esperado, por exemplo string em vez de número.

ObjectId comparado com uma string sem conversão.

Data armazenada em formatos inconsistentes.

Filtro amplo demais em update/delete.

Índice ausente para uma consulta crítica.

URI ou credencial incorreta.

Permissão insuficiente no usuário do banco.

FIM DA APOSTILA

MongoDB + Python

Material estruturado para estudo progressivo, prática e desenvolvimento de projetos.

Fontes técnicas consultadas para esta edição: documentação oficial do MongoDB, documentação do driver PyMongo e cursos oficiais do MongoDB University.